

Cell 1: Install Dependencies

```
!pip install torchtext nltk
```

Collecting torchtext

Downloading torchtext-0.18.0-cp311-cp311-

manylinux1_x86_64.whl.metadata (7.9 kB)

Requirement already satisfied: nltk in /usr/local/lib/python3.11/dist-packages (3.9.1)

Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from torchtext) (4.67.1)

Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from torchtext) (2.32.3)

Requirement already satisfied: torch>=2.3.0 in /usr/local/lib/python3.11/dist-packages (from torchtext) (2.6.0+cu124)

Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from torchtext) (2.0.2)

Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk) (8.1.8)

Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk) (1.4.2)

Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk) (2024.11.6)

Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext) (3.18.0)

Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext) (4.12.2)

Requirement already satisfied: networkx in /usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext) (3.4.2)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext) (3.1.6)

Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext) (2025.3.0)

Collecting nvidia-cuda-nvrtc-cu12==12.4.127 (from torch>=2.3.0->torchtext)

Downloading nvidia_cuda_nvrtc_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)

Collecting nvidia-cuda-runtime-cu12==12.4.127 (from torch>=2.3.0->torchtext)

Downloading nvidia_cuda_runtime_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.5 kB)

Collecting nvidia-cuda-cupti-cu12==12.4.127 (from torch>=2.3.0->torchtext)

Downloading nvidia_cuda_cupti_cu12-12.4.127-py3-none-manylinux2014_x86_64.whl.metadata (1.6 kB)

Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch>=2.3.0->torchtext)

```
Downloading nvidia_cudnn_cu12-9.1.0.70-py3-none-  
manylinux2014_x86_64.whl.metadata (1.6 kB)  
Collecting nvidia-cublas-cu12==12.4.5.8 (from torch>=2.3.0->torchtext)  
Downloading nvidia_cublas_cu12-12.4.5.8-py3-none-  
manylinux2014_x86_64.whl.metadata (1.5 kB)  
Collecting nvidia-cufft-cu12==11.2.1.3 (from torch>=2.3.0->torchtext)  
Downloading nvidia_cufft_cu12-11.2.1.3-py3-none-  
manylinux2014_x86_64.whl.metadata (1.5 kB)  
Collecting nvidia-curand-cu12==10.3.5.147 (from torch>=2.3.0-  
>torchtext)  
Downloading nvidia_curand_cu12-10.3.5.147-py3-none-  
manylinux2014_x86_64.whl.metadata (1.5 kB)  
Collecting nvidia-cusolver-cu12==11.6.1.9 (from torch>=2.3.0-  
>torchtext)  
Downloading nvidia_cusolver_cu12-11.6.1.9-py3-none-  
manylinux2014_x86_64.whl.metadata (1.6 kB)  
Collecting nvidia-cusparselt-cu12==0.6.2 (from torch>=2.3.0-  
>torchtext)  
Requirement already satisfied: nvidia-cusparselt-cu12==0.6.2 in  
/usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext)  
(0.6.2)  
Requirement already satisfied: nvidia-nccl-cu12==2.21.5 in  
/usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext)  
(2.21.5)  
Requirement already satisfied: nvidia-nvtx-cu12==12.4.127 in  
/usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext)  
(12.4.127)  
Collecting nvidia-nvjitlink-cu12==12.4.127 (from torch>=2.3.0-  
>torchtext)  
Downloading nvidia_nvjitlink_cu12-12.4.127-py3-none-  
manylinux2014_x86_64.whl.metadata (1.5 kB)  
Requirement already satisfied: triton==3.2.0 in  
/usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext)  
(3.2.0)  
Requirement already satisfied: sympy==1.13.1 in  
/usr/local/lib/python3.11/dist-packages (from torch>=2.3.0->torchtext)  
(1.13.1)  
Requirement already satisfied: mpmath<1.4,>=1.1.0 in  
/usr/local/lib/python3.11/dist-packages (from sympy==1.13.1-  
>torch>=2.3.0->torchtext) (1.3.0)  
Requirement already satisfied: charset-normalizer<4,>=2 in  
/usr/local/lib/python3.11/dist-packages (from requests->torchtext)  
(3.4.1)  
Requirement already satisfied: idna<4,>=2.5 in  
/usr/local/lib/python3.11/dist-packages (from requests->torchtext)  
(3.10)  
Requirement already satisfied: urllib3<3,>=1.21.1 in
```

```

/usr/local/lib/python3.11/dist-packages (from requests->torchtext)
(2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.11/dist-packages (from requests->torchtext)
(2025.1.31)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.11/dist-packages (from jinja2->torch>=2.3.0-
>torchtext) (3.0.2)
Downloading torchtext-0.18.0-cp311-cp311-manylinux1_x86_64.whl (2.0
MB)
----- 2.0/2.0 MB 30.8 MB/s eta
0:00:00
anylinux2014_x86_64.whl (363.4 MB)
----- 363.4/363.4 MB 4.2 MB/s eta
0:00:00
anylinux2014_x86_64.whl (13.8 MB)
----- 13.8/13.8 MB 40.5 MB/s eta
0:00:00
anylinux2014_x86_64.whl (24.6 MB)
----- 24.6/24.6 MB 29.7 MB/s eta
0:00:00
e_cul2-12.4.127-py3-none-manylinux2014_x86_64.whl (883 kB)
----- 883.7/883.7 kB 40.2 MB/s eta
0:00:00
anylinux2014_x86_64.whl (664.8 MB)
----- 664.8/664.8 MB 2.2 MB/s eta
0:00:00
anylinux2014_x86_64.whl (211.5 MB)
----- 211.5/211.5 MB 4.9 MB/s eta
0:00:00
anylinux2014_x86_64.whl (56.3 MB)
----- 56.3/56.3 MB 10.2 MB/s eta
0:00:00
anylinux2014_x86_64.whl (127.9 MB)
----- 127.9/127.9 MB 6.8 MB/s eta
0:00:00
anylinux2014_x86_64.whl (207.5 MB)
----- 207.5/207.5 MB 4.9 MB/s eta
0:00:00
anylinux2014_x86_64.whl (21.1 MB)
----- 21.1/21.1 MB 50.3 MB/s eta
0:00:00
e-cul2, nvidia-cuda-nvrtc-cul2, nvidia-cuda-cupti-cul2, nvidia-cublas-
cul2, nvidia-cuspars-cul2, nvidia-cudnn-cul2, nvidia-cusolver-cul2,
torchtext
Attempting uninstall: nvidia-nvjitlink-cul2
Found existing installation: nvidia-nvjitlink-cul2 12.5.82
Uninstalling nvidia-nvjitlink-cul2-12.5.82:
Successfully uninstalled nvidia-nvjitlink-cul2-12.5.82

```

```
Attempting uninstall: nvidia-curand-cu12
Found existing installation: nvidia-curand-cu12 10.3.6.82
Uninstalling nvidia-curand-cu12-10.3.6.82:
Successfully uninstalled nvidia-curand-cu12-10.3.6.82
Attempting uninstall: nvidia-cufft-cu12
Found existing installation: nvidia-cufft-cu12 11.2.3.61
Uninstalling nvidia-cufft-cu12-11.2.3.61:
Successfully uninstalled nvidia-cufft-cu12-11.2.3.61
Attempting uninstall: nvidia-cuda-runtime-cu12
Found existing installation: nvidia-cuda-runtime-cu12 12.5.82
Uninstalling nvidia-cuda-runtime-cu12-12.5.82:
Successfully uninstalled nvidia-cuda-runtime-cu12-12.5.82
Attempting uninstall: nvidia-cuda-nvrtc-cu12
Found existing installation: nvidia-cuda-nvrtc-cu12 12.5.82
Uninstalling nvidia-cuda-nvrtc-cu12-12.5.82:
Successfully uninstalled nvidia-cuda-nvrtc-cu12-12.5.82
Attempting uninstall: nvidia-cuda-cupti-cu12
Found existing installation: nvidia-cuda-cupti-cu12 12.5.82
Uninstalling nvidia-cuda-cupti-cu12-12.5.82:
Successfully uninstalled nvidia-cuda-cupti-cu12-12.5.82
Attempting uninstall: nvidia-cublas-cu12
Found existing installation: nvidia-cublas-cu12 12.5.3.2
Uninstalling nvidia-cublas-cu12-12.5.3.2:
Successfully uninstalled nvidia-cublas-cu12-12.5.3.2
Attempting uninstall: nvidia-cusparse-cu12
Found existing installation: nvidia-cusparse-cu12 12.5.1.3
Uninstalling nvidia-cusparse-cu12-12.5.1.3:
Successfully uninstalled nvidia-cusparse-cu12-12.5.1.3
Attempting uninstall: nvidia-cudnn-cu12
Found existing installation: nvidia-cudnn-cu12 9.3.0.75
Uninstalling nvidia-cudnn-cu12-9.3.0.75:
Successfully uninstalled nvidia-cudnn-cu12-9.3.0.75
Attempting uninstall: nvidia-cusolver-cu12
Found existing installation: nvidia-cusolver-cu12 11.6.3.83
Uninstalling nvidia-cusolver-cu12-11.6.3.83:
Successfully uninstalled nvidia-cusolver-cu12-11.6.3.83
Successfully installed nvidia-cublas-cu12-12.4.5.8 nvidia-cuda-cupti-
cu12-12.4.127 nvidia-cuda-nvrtc-cu12-12.4.127 nvidia-cuda-runtime-
cu12-12.4.127 nvidia-cudnn-cu12-9.1.0.70 nvidia-cufft-cu12-11.2.1.3
nvidia-curand-cu12-10.3.5.147 nvidia-cusolver-cu12-11.6.1.9 nvidia-
cusparse-cu12-12.3.1.170 nvidia-nvjitlink-cu12-12.4.127 torchtext-
0.18.0
```

Cell 2: Import Libraries

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import matplotlib.pyplot as plt
```

```
import nltk
import random
import re
import string
import numpy as np
from torch.utils.data import Dataset, DataLoader
from nltk.tokenize import word_tokenize
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

```
True
```

```
# Cell 3: Load Dataset (Cornell Movie Dialogs Corpus)
```

```
!wget
https://www.cs.cornell.edu/~cristian/data/cornell_movie_dialogs_corpus
.zip
!unzip -o cornell_movie_dialogs_corpus.zip
```

```
--2025-03-31 11:18:39--
```

```
https://www.cs.cornell.edu/~cristian/data/cornell_movie_dialogs_corpus
.zip
```

```
Resolving www.cs.cornell.edu (www.cs.cornell.edu)... 132.236.207.53
```

```
Connecting to www.cs.cornell.edu (www.cs.cornell.edu)|
```

```
132.236.207.53|:443... connected.
```

```
HTTP request sent, awaiting response... 200 OK
```

```
Length: 9916637 (9.5M) [application/zip]
```

```
Saving to: 'cornell_movie_dialogs_corpus.zip'
```

```
cornell_movie_dialo 100%[=====>]   9.46M  14.4MB/s   in
0.7s
```

```
2025-03-31 11:18:40 (14.4 MB/s) - 'cornell_movie_dialogs_corpus.zip'
saved [9916637/9916637]
```

```
Archive:  cornell_movie_dialogs_corpus.zip
```

```
creating: cornell movie-dialogs corpus/
```

```
inflating: cornell movie-dialogs corpus/.DS_Store
```

```
creating: __MACOSX/
```

```
creating: __MACOSX/cornell movie-dialogs corpus/
```

```
inflating: __MACOSX/cornell movie-dialogs corpus/._DS_Store
```

```
inflating: cornell movie-dialogs corpus/chameleons.pdf
```

```
inflating: __MACOSX/cornell movie-dialogs corpus/._chameleons.pdf
```

```
inflating: cornell movie-dialogs
```

```
corpus/movie_characters_metadata.txt
```

```
inflating: cornell movie-dialogs corpus/movie_conversations.txt
```

```
inflating: cornell movie-dialogs corpus/movie_lines.txt
```

```
inflating: cornell movie-dialogs corpus/movie_titles_metadata.txt
```

```
inflating: cornell movie-dialogs corpus/raw_script_urls.txt
```

```
inflating: cornell movie-dialogs corpus/README.txt
inflating: __MACOSX/cornell movie-dialogs corpus/._README.txt
```

```
# Cell 4: Parse Dataset
```

```
def load_conversations():
    with open("cornell movie-dialogs corpus/movie_lines.txt",
encoding='utf-8', errors='ignore') as f:
        lines = f.read().split('\n')
        id2line = {}
        for line in lines:
            parts = line.split(" +++$+++ ")
            if len(parts) == 5:
                id2line[parts[0]] = parts[4]

    with open("cornell movie-dialogs corpus/movie_conversations.txt",
encoding='utf-8', errors='ignore') as f:
        conversations = f.read().split('\n')

    pairs = []
    for conv in conversations:
        try:
            ids = eval(conv.split(" +++$+++ ")[-1])
            for i in range(len(ids) - 1):
                input_line = id2line[ids[i]]
                target_line = id2line[ids[i + 1]]
                pairs.append((input_line, target_line))
        except:
            pass
    return pairs

pairs = load_conversations()
print("Sample data pair:\n", pairs[0])
print("Total pairs:", len(pairs))
```

```
Sample data pair:
```

```
('Can we make this quick? Roxanne Korrine and Andrew Barrett are
having an incredibly horrendous public break- up on the quad.
Again.', "Well, I thought we'd start with pronunciation, if that's
okay with you.")
```

```
Total pairs: 221616
```

```
import nltk
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

```
True
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```

# Calculate sentence lengths
input_lengths = [len(word_tokenize(pair[0])) for pair in pairs]
target_lengths = [len(word_tokenize(pair[1])) for pair in pairs]

# Create histograms
plt.figure(figsize=(12, 5))

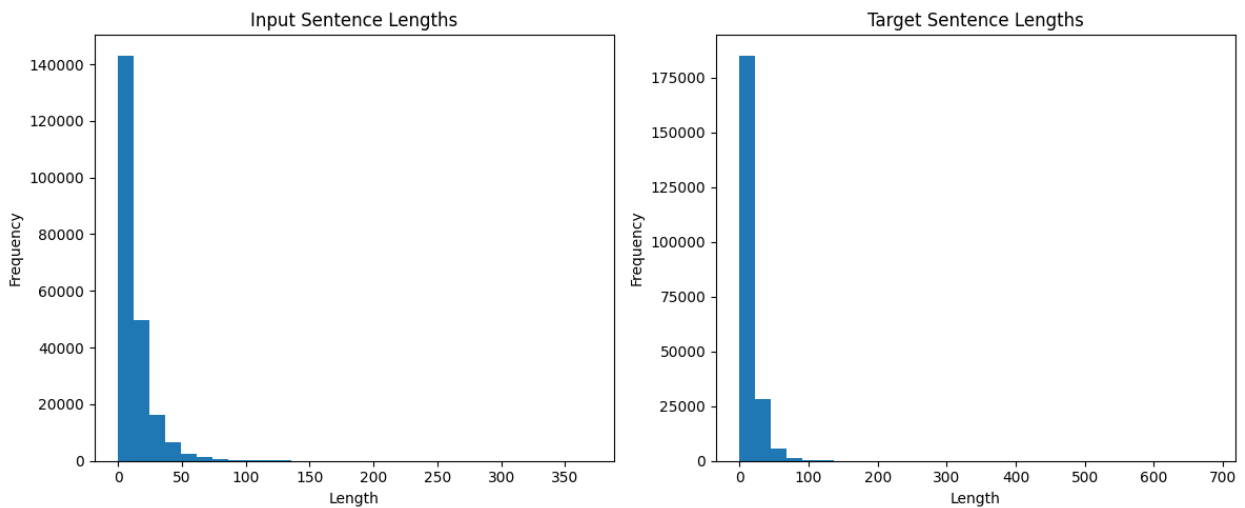
plt.subplot(1, 2, 1)
plt.hist(input_lengths, bins=30)
plt.title('Input Sentence Lengths')
plt.xlabel('Length')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
plt.hist(target_lengths, bins=30)
plt.title('Target Sentence Lengths')
plt.xlabel('Length')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()

# Further analysis (optional)
print("Average input length:", np.mean(input_lengths))
print("Average target length:", np.mean(target_lengths))
print("Max input length:", np.max(input_lengths))
print("Max target length:", np.max(target_lengths))

```



```

Average input length: 13.314449317738791
Average target length: 13.816299364666811
Max input length: 370
Max target length: 684

```

```

# Cell 5: Preprocessing
def clean_text(text):
    text = text.lower()
    text = re.sub(r"^[a-zA-Z0-9]+", " ", text)
    return text.strip()

pairs = [(clean_text(q), clean_text(a)) for q, a in pairs[:10000]] #
limit to 10k pairs for speed

# Cell 6: Tokenization and Vocabulary
all_words = []
for pair in pairs:
    all_words += word_tokenize(pair[0])
    all_words += word_tokenize(pair[1])

all_words = sorted(set(all_words))
word2idx = {word: idx+4 for idx, word in enumerate(all_words)}
word2idx["<PAD>"] = 0
word2idx["<SOS>"] = 1
word2idx["<EOS>"] = 2
word2idx["<UNK>"] = 3
idx2word = {idx: word for word, idx in word2idx.items()}

def encode_sentence(sent):
    return [word2idx.get(word, word2idx["<UNK>"]) for word in
word_tokenize(sent)] + [word2idx["<EOS>"]]

import numpy as np
import matplotlib.pyplot as plt

# Create histograms
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.hist(input_lengths, bins=30)
plt.title('Input Sentence Lengths')
plt.xlabel('Length')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
plt.hist(target_lengths, bins=30)
plt.title('Target Sentence Lengths')
plt.xlabel('Length')
plt.ylabel('Frequency')

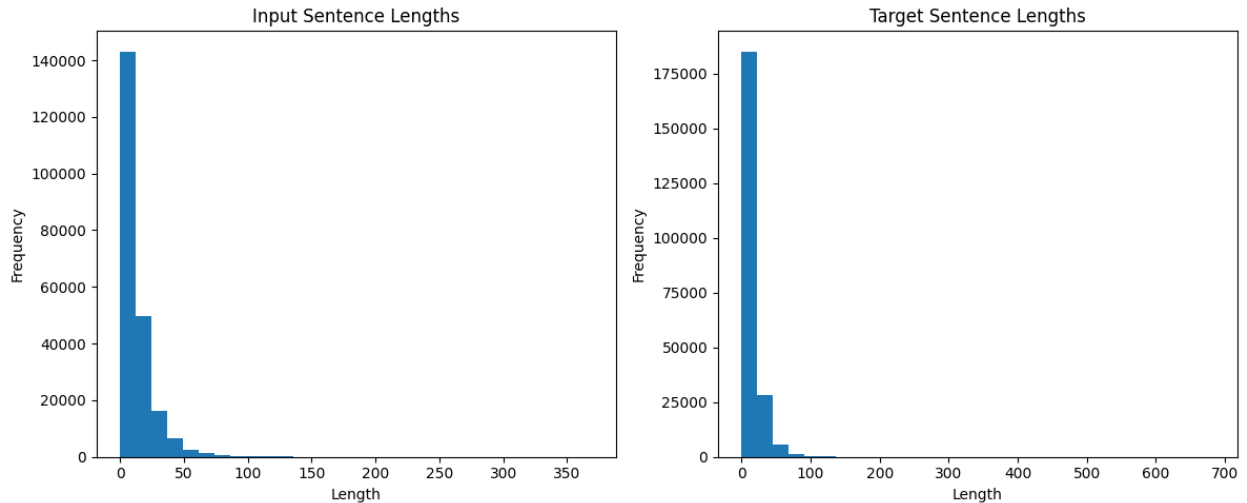
plt.tight_layout()
plt.show()

# Further analysis (optional)
print("Average input length:", np.mean(input_lengths))

```



```
print("Average target length:", np.mean(target_lengths))
print("Max input length:", np.max(input_lengths))
print("Max target length:", np.max(target_lengths))
```



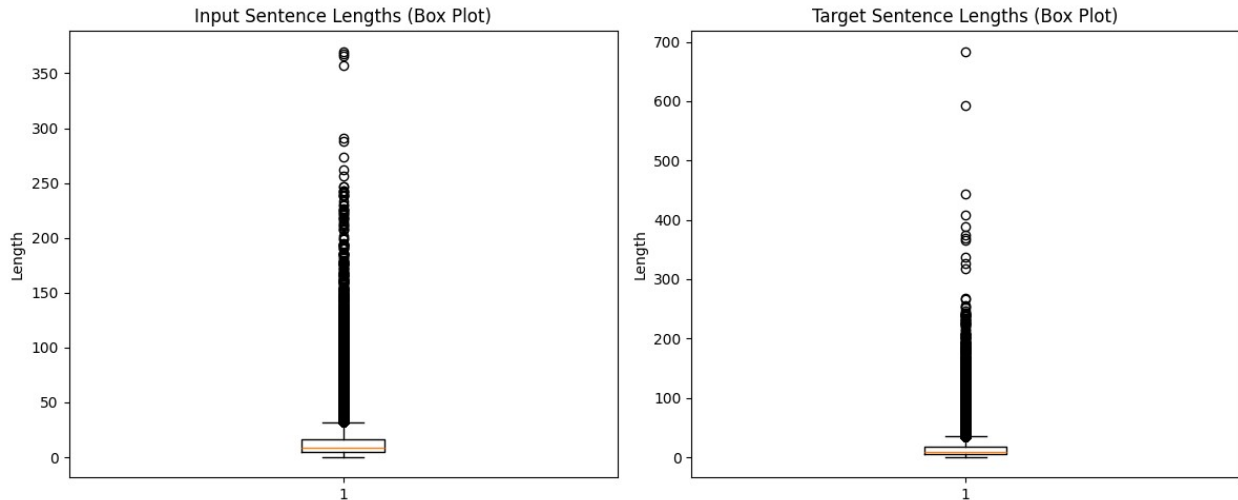
```
Average input length: 13.314449317738791
Average target length: 13.816299364666811
Max input length: 370
Max target length: 684
```

```
# Box plots for a more detailed view of the distribution
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
plt.boxplot(input_lengths)
plt.title('Input Sentence Lengths (Box Plot)')
plt.ylabel('Length')
```

```
plt.subplot(1, 2, 2)
plt.boxplot(target_lengths)
plt.title('Target Sentence Lengths (Box Plot)')
plt.ylabel('Length')
```

```
plt.tight_layout()
plt.show()
```



Cell 7: Dataset Preparation

```
class ChatDataset(Dataset):
    def __init__(self, pairs):
        self.pairs = pairs

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        q, a = self.pairs[idx]
        return torch.tensor(encode_sentence(q)),
        torch.tensor(encode_sentence(a))

def collate_fn(batch):
    input_batch, target_batch = zip(*batch)
    input_batch = nn.utils.rnn.pad_sequence(input_batch,
        batch_first=True, padding_value=0)
    target_batch = nn.utils.rnn.pad_sequence(target_batch,
        batch_first=True, padding_value=0)
    return input_batch, target_batch

dataset = ChatDataset(pairs)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True,
    collate_fn=collate_fn)
```

Cell 8: Seq2Seq Model with Attention

```
class Encoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)

    def forward(self, x):
        embedded = self.embedding(x)
```

```

        outputs, (h, c) = self.lstm(embedded)
        return outputs, h, c

class Decoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x, h, c):
        x = self.embedding(x)
        outputs, (h, c) = self.lstm(x, (h, c))
        return self.fc(outputs), h, c

class Seq2Seq(nn.Module):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, src, tgt):
        encoder_outputs, h, c = self.encoder(src)
        output, _, _ = self.decoder(tgt[:, :-1], h, c)
        return output

# Cell 9: Training
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

vocab_size = len(word2idx)
embed_size = 256
hidden_size = 512

encoder = Encoder(vocab_size, embed_size, hidden_size).to(device)
decoder = Decoder(vocab_size, embed_size, hidden_size).to(device)
model = Seq2Seq(encoder, decoder).to(device)

criterion = nn.CrossEntropyLoss(ignore_index=0)
optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 50
losses = []

for epoch in range(epochs):
    total_loss = 0
    for input_seq, target_seq in dataloader:
        input_seq, target_seq = input_seq.to(device),
        target_seq.to(device)
        output = model(input_seq, target_seq)

```

```
        loss = criterion(output.view(-1, vocab_size), target_seq[:,
1:].reshape(-1))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

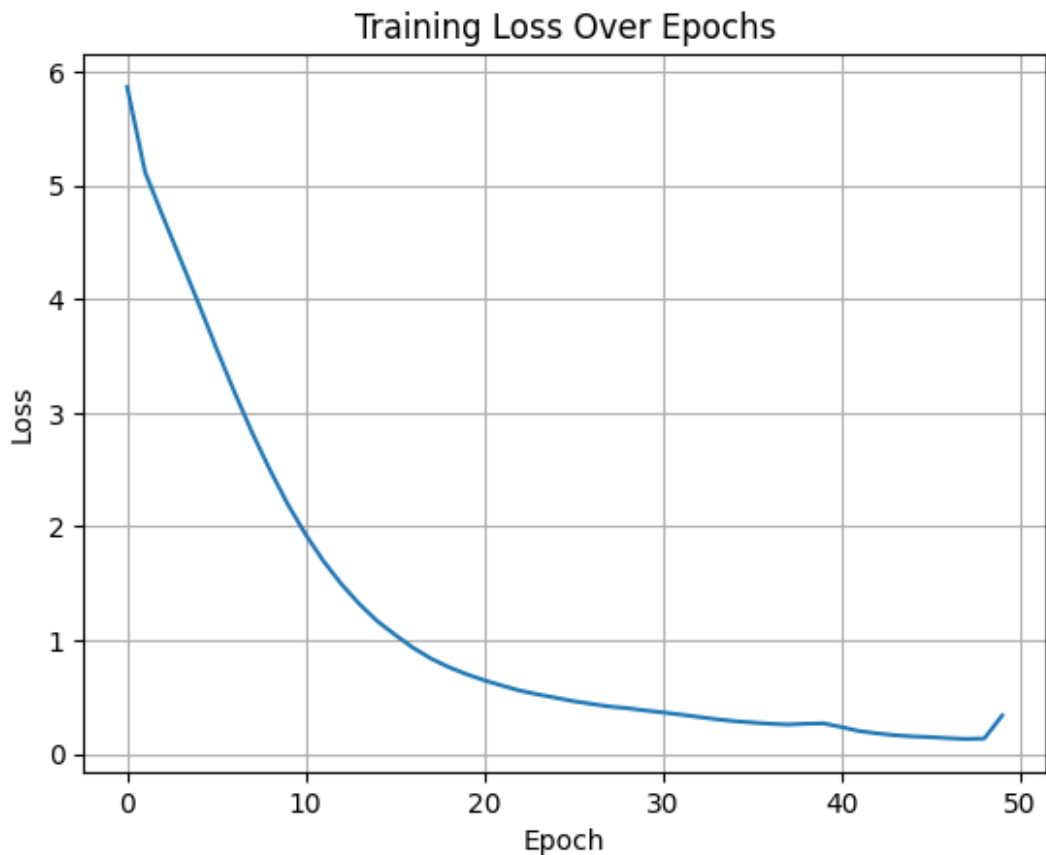
        total_loss += loss.item()
    avg_loss = total_loss / len(dataloader)
    losses.append(avg_loss)
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}")
```

```
Epoch [1/50], Loss: 5.8653
Epoch [2/50], Loss: 5.1170
Epoch [3/50], Loss: 4.7260
Epoch [4/50], Loss: 4.3440
Epoch [5/50], Loss: 3.9592
Epoch [6/50], Loss: 3.5649
Epoch [7/50], Loss: 3.1847
Epoch [8/50], Loss: 2.8244
Epoch [9/50], Loss: 2.4956
Epoch [10/50], Loss: 2.1935
Epoch [11/50], Loss: 1.9285
Epoch [12/50], Loss: 1.6941
Epoch [13/50], Loss: 1.4923
Epoch [14/50], Loss: 1.3193
Epoch [15/50], Loss: 1.1681
Epoch [16/50], Loss: 1.0480
Epoch [17/50], Loss: 0.9342
Epoch [18/50], Loss: 0.8395
Epoch [19/50], Loss: 0.7639
Epoch [20/50], Loss: 0.7010
Epoch [21/50], Loss: 0.6470
Epoch [22/50], Loss: 0.5999
Epoch [23/50], Loss: 0.5561
Epoch [24/50], Loss: 0.5231
Epoch [25/50], Loss: 0.4934
Epoch [26/50], Loss: 0.4636
Epoch [27/50], Loss: 0.4397
Epoch [28/50], Loss: 0.4164
Epoch [29/50], Loss: 0.4020
Epoch [30/50], Loss: 0.3819
Epoch [31/50], Loss: 0.3644
Epoch [32/50], Loss: 0.3454
Epoch [33/50], Loss: 0.3249
Epoch [34/50], Loss: 0.3047
Epoch [35/50], Loss: 0.2879
Epoch [36/50], Loss: 0.2756
Epoch [37/50], Loss: 0.2651
Epoch [38/50], Loss: 0.2580
Epoch [39/50], Loss: 0.2648
```

```
Epoch [40/50], Loss: 0.2683
Epoch [41/50], Loss: 0.2357
Epoch [42/50], Loss: 0.2003
Epoch [43/50], Loss: 0.1806
Epoch [44/50], Loss: 0.1639
Epoch [45/50], Loss: 0.1537
Epoch [46/50], Loss: 0.1471
Epoch [47/50], Loss: 0.1379
Epoch [48/50], Loss: 0.1310
Epoch [49/50], Loss: 0.1352
Epoch [50/50], Loss: 0.3379
```

```
# Cell 10: Plot Loss
```

```
plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss Over Epochs")
plt.grid(True)
plt.show()
```



```
# Cell 11: Save Model
```

```
torch.save(model.state_dict(), "chatbot_model.pth")
```

```

# Cell 12: Inference
def chat(model, input_text):
    model.eval()
    input_seq =
torch.tensor(encode_sentence(clean_text(input_text))).unsqueeze(0).to(
device)
    encoder_outputs, h, c = model.encoder(input_seq)

    tgt = torch.tensor([[word2idx["<SOS>"]]]).to(device)
    response = []

    for _ in range(20):
        out, h, c = model.decoder(tgt, h, c)
        pred_id = out.argmax(2)[: , -1].item()
        if pred_id == word2idx["<EOS>"]:
            break
        response.append(idx2word.get(pred_id, "<UNK>"))
        tgt = torch.cat([tgt, torch.tensor([[pred_id]]).to(device)],
dim=1)

    return " ".join(response)

print(chat(model, "Hello, how are you?"))
print(chat(model, "What is your name?"))

```

do you will be given to any of out
in the course of an evening i think we re in one of a very kind of
weather man i

prompt: download trained model to local machine

```

from google.colab import files
files.download('chatbot_model.pth')

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

```

prompt: evaluate the model

Cell 13: Evaluate the model (example interactions)

```

# Load the saved model
model.load_state_dict(torch.load("chatbot_model.pth"))
model.to(device)
model.eval()

```

Example interactions

```

test_sentences = [
    "Hello",
    "How are you doing?",

```

```
    "What's your favorite movie?",  
    "Tell me a joke.",  
    "Goodbye"  
]
```

```
for sentence in test_sentences:  
    response = chat(model, sentence)  
    print(f"User: {sentence}")  
    print(f"Bot: {response}")  
    print("---")
```

User: Hello

Bot: to be very simple box i m a huge strong fellow

User: How are you doing?

Bot: i m a negro

User: What's your favorite movie?

Bot: your wife may be right to the fact gracefully out of the way we
can go double a fire dead

User: Tell me a joke.

Bot: love changes only and double an affair and jack went to that
feeling as a fire press out that s

User: Goodbye

Bot: you want to get some food old women some old trying to education
to hold it away
